

Production-ready security project architecture for junior engineers ▾



Role: Bạn là một Principal Security Architect và DevSecOps Expert tại một công ty công nghệ lớn.

Context: Tôi là một sinh viên mới ra trường nhưng muốn xây dựng một đồ án/portfolio project thể hiện được tư duy và kỹ năng của một Kỹ sư Bảo mật/DevSecOps có 3-4 năm kinh

Show more

Orchestrated production PKI architecture leveraging existing research into differenti...

- 🕒 So the student wants to take their existing PKI project—which already handles OCSP soft-fail and trusted-CA MITM scenarios—and expand it into a production-grade open-source tool that's easy to deploy with just git clone and docker-compose. They're looking for something more substantial with better tooling, and they prefer communicating in Vietnamese with a direct, casual tone. The smart play here is to merge their existing research into Option 1 (Enterprise PKI & Secret Management) by turning their OCSP soft-fail measurement work into a built-in revocation enforcement validator and conformance test harness. That's what makes it stand out—not just another Vault-in-docker template, but a PKI platform paired with a continuous trust-validation harness that actually proves revocation works. It reuses their existing work instead of discarding it, and that's the real differentiator. I should be honest with them: basic Vault docker-compose repos are everywhere; their competitive edge is the measurement harness they've already built. They want architecture diagrams (as text descriptions), directory structure, data flow showing how security tools interact, and step-by-step setup from scratch. For delivery, this is a substantial design document they'll reference and iterate on, so I'll keep it inline in chat rather than creating a separate file—faster for discussion and

refinement. I'll assume the project name is obvious from context and move straight into designing the architecture properly.

Now I'm settling on a project name and defining the core value proposition. The insight here is that most SMEs run internal PKI without proper revocation enforcement or inventory—existing tools like Vault and step-ca issue certs but don't verify clients actually respect revocation. I'm structuring this as three planes: an issuance layer using Vault or step-ca with Terraform provisioning and ACME endpoints so workloads can request certs automatically, a revocation and enforcement layer, and monitoring. I'll store drift detection via an osquery-like script that hashes root certificates on endpoints and alerts when new CAs appear. The DevSecOps pipeline runs gitleaks, trivy, hadolint, tfsec/checkov, and SonarQube with SBOM generation and cosign signing, plus conftest/OPA policies. The integration test spins up the entire stack in CI and verifies that revocation detection completes within a target threshold — essentially making the security property itself testable in the pipeline. The architecture layers the control plane with Vault's PKI and transit engines, backed by either Consul or integrated Raft storage. For the seal strategy, I'm distinguishing between the demo setup (Shamir with a local dev key) and production (auto-unseal via a separate KMS or dedicated seal Vault), since that nuance signals deeper security thinking. The issuance chain flows from an offline Vault PKI root through an online intermediate to an ACME endpoint (Vault 1.14+ native support), which clients like Traefik, cert-manager, or step-cli consume. Status services include Vault's built-in OCSP responder for ACME/PKI and CRL distribution via nginx, plus a CRL publishing job. The assurance layer adds the revocation probe, trust-store-drift agent, and a CT log monitor watching crt.sh for external certificate issuance.

The observability stack pulls metrics into Prometheus and Grafana, logs into Loki or Elastic, and security events into Wazuh. At the edge, Traefik enforces mTLS on admin routes and uses Let's Encrypt (staging by default) for the public domain. The demo app is a small Go or Python service that authenticates with a Vault-issued client certificate to showcase machine identity in action.

I need to lay out the directory structure as a concrete tree, then map the data flows: bootstrap, cert issuance, revocation, the assurance loop, alerting, and CI integration. The roadmap should be phased over 5–6 weeks with clear milestones and time estimates, probably six phases. I should also flag the scope creep risk upfront—they shouldn't attempt the full Vault plus Kubernetes plus everything stack in one go. The real impact comes from the README, a demo GIF, and the assurance metric itself. I'll keep the license, CONTRIBUTING, security.md, and docs/adr/ sections tight. The whole thing will be dense with headers and code blocks to manage length. I'll write in a casual, direct tone matching their style without overdoing the informal register.

I should verify a few details: Vault's ACME support came in 1.14 (2023), and the built-in OCSP responder landed around the same time. I'll hedge slightly on version

specifics. Also worth flagging that Vault switched to BUSL licensing in August 2023, so mentioning OpenBao as a license-free alternative is a solid senior-level signal.

Show less

☑ Done

Chốt luôn hướng: **Option 1 (PKI + Secret Management)**, nhưng không phải bản "docker-compose lên Vault" như 500 repo khác trên GitHub. Nói thẳng: Vault-in-docker-compose là repo rác, recruiter nhìn 5 giây là biết bạn copy tutorial của HashiCorp.

Thứ khiến project của bạn khác biệt là **cái research CYB 260 bạn đã có**: OCSP soft-fail và trusted-CA MITM. Không ai đóng gói cái đó thành product cả. Mọi PKI tool trên thị trường (step-ca, Vault PKI, cert-manager) đều **cấp** cert — không tool nào **chứng minh** rằng client thật sự enforce revocation. Đó là gap thật, và bạn đã có data đo được nó.

Nên project = **PKI platform + continuous assurance harness**. Biến RQ1/RQ2 từ "bài lab" thành "control plane có Prometheus metric".

1. Định danh project

Tên đề xuất: `pki-sentinel` (alt: `trustforge`, `certmesh`)

One-liner:

Self-hosted internal PKI for SMEs — issues short-lived certificates via ACME, and continuously *proves* that your clients actually reject revoked ones.

Vấn đề thật của SME (viết đúng cái này vào README "Why I built this"):

- Internal service dùng self-signed cert, hết hạn lúc 2h sáng thứ 7 → downtime.
- Không ai biết trong tổ chức có bao nhiêu root CA đang được trust trên endpoint.
- Có deploy revocation nhưng chưa từng test → soft-fail, revoke xong cert vẫn dùng được.

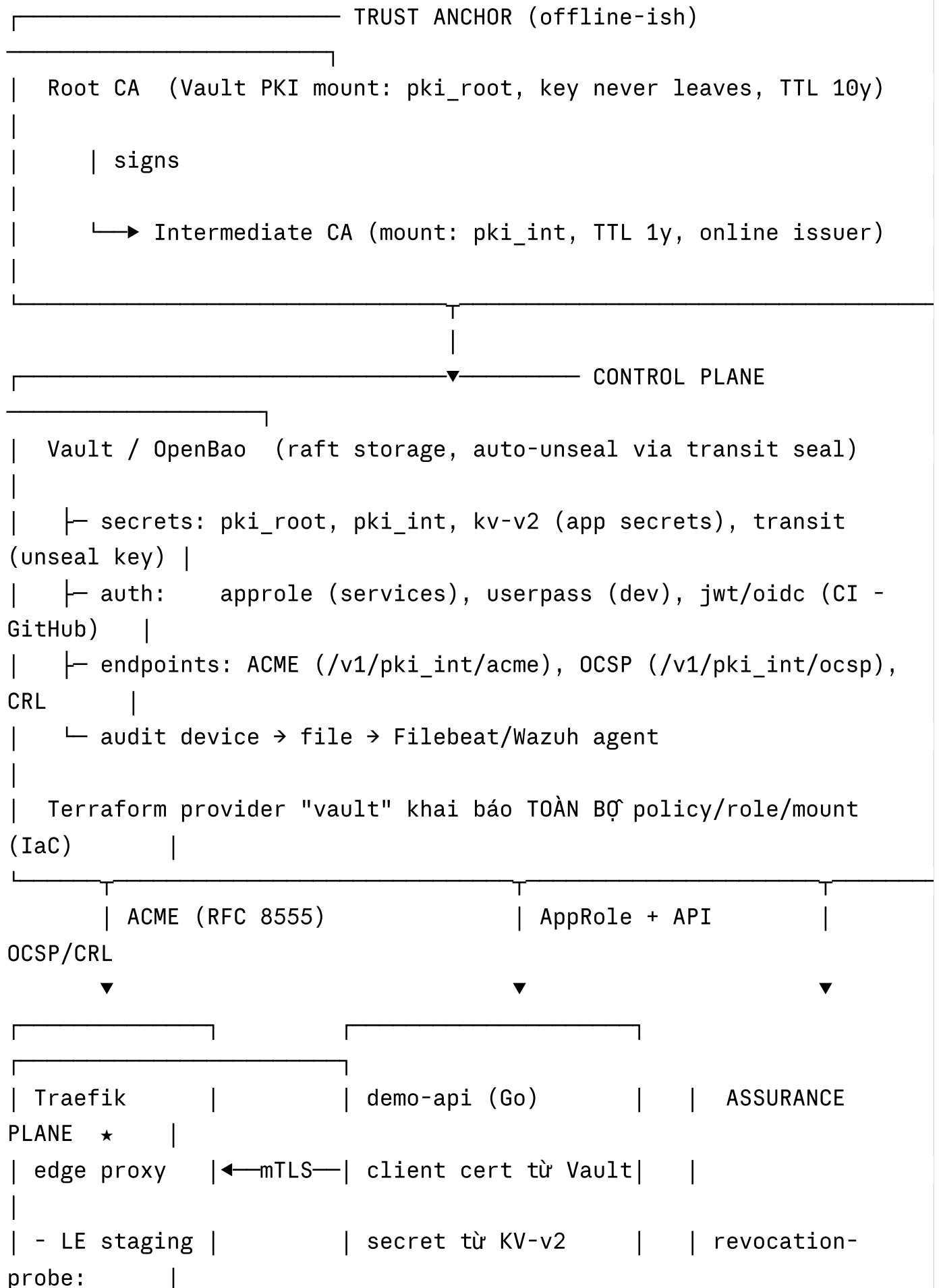
3 control plane:

Plane	Vai trò	Tools
Issuance	Cấp/renew cert tự động	Vault PKI (Root + Intermediate), ACME endpoint, Traefik, cert-manager (optional k8s)
Assurance (điểm ăn tiền)	Đo xem revocation/trust có thật sự hoạt động	revocation-probe, truststore-drift-agent, ct-watcher
Governance	Log, alert, inventory, audit	Prometheus + Grafana, Wazuh/ELK, Alertmanager → Slack/Telegram

Lưu ý license: Vault đã chuyển sang BUSL. Nên hỗ trợ **OpenBao** (fork Apache-2.0) qua biến env `VAULT_IMAGE`. Chi tiết nhỏ này là signal rất mạnh — chứng tỏ bạn biết legal constraint của enterprise, không chỉ biết gõ lệnh.

2. Architecture Diagram (mô tả để vẽ lại bằng draw.io / Excalidraw)

Vẽ 5 khối ngang, mũi tên đi từ trái sang phải, plane Assurance vẽ màu đỏ/cam để nổi bật.





```
| Wazuh manager (hoặc ELK) ← audit log Vault, Traefik access,  
agent events |
```

```
CI/CD (GitHub Actions)
```

```
| gitleaks → tflint/checkov → hadolint → build → trivy(image+fs) →  
syft SBOM |
```

```
| → cosign sign → INTEGRATION TEST (dựng full stack, assert  
t_detect < 15s) |
```

```
| → gate: CRITICAL vuln = fail → push GHCR
```

Cái ★ là thứ recruiter chưa từng thấy trong portfolio nào. Đừng để nó ở góc diagram — để giữa.

3. Directory Structure

```

pki-sentinel/
├─ README.md                # có badge, GIF demo, kiến trúc,
quick start 3 lệnh
├─ SECURITY.md              # threat model + responsible
disclosure
├─ CONTRIBUTING.md
├─ LICENSE                  # Apache-2.0
├─ Makefile                 # make up / make down / make test
/ make demo-revoke
├─ .env.example
├─ docker-compose.yml       # stack chính
├─ docker-compose.observability.yml
|
├─ docs/
|   └─ architecture.md
|   └─ threat-model.md     # STRIDE cho từng plane
|   └─ adr/                 # Architecture Decision Records ←
senior signal
|   |   └─ 0001-vault-vs-step-ca.md
|   |   └─ 0002-auto-unseal-tradeoffs.md
|   |   └─ 0003-short-lived-certs-over-revocation.md
|   └─ runbooks/
|       └─ root-ca-compromise.md
|       └─ vault-seal-recovery.md
|       └─ rotate-intermediate.md
|   └─ benchmarks/        # ← research CYB 260 của bạn sống
ở đây
|       └─ ocsf-softfail-thresholds.md
|       └─ images/
|
├─ terraform/
|   └─ bootstrap/          # tạo mount, root, intermediate
|   └─ policies/           # HCL policy files
|   └─ roles/              # pki roles + approle
|       └─ modules/
|           └─ pki-hierarchy/

```



```

|       └─ app-identity/
|
|─ ansible/                                # deploy lên VM thật (không chỉ
docker)
|   └─ playbooks/site.yml
|       └─ roles/{vault,traefik,wazuh-agent,truststore-agent}/
|
|─ services/
|   └─ revocation-probe/                  # Go - trái tim của project
|       └─ cmd/probe/main.go
|           └─ internal/{issuer,clientprofiles,metrics,scheduler}/
|               └─ profiles.yaml          # curl-hardfail, curl-softfail,
openssl s_client, go-tls, python-requests, chromium-headless
|           └─ Dockerfile
|   └─ truststore-drift-agent/            # Go/Python - hash root store, so
với baseline
|       └─ ct-watcher/
|           └─ demo-api/                  # app mẫu dùng mTLS + secret từ
Vault
|
|─ observability/
|   └─ prometheus/{prometheus.yml,rules/pki-slo.yml}
|   └─ grafana/dashboards/{cert-inventory.json,revocation-
slo.json,trust-drift.json}
|   └─ alertmanager/config.yml
|       └─ wazuh/{decoders,rules}/vault_audit.xml    # custom decoder <
rất ít người làm
|
|─ policy/                                # OPA/Conftest
|   └─
opa/{no_wildcard_san.rego,max_cert_ttl.rego,require_ocsp_aia.rego}
|
|─ tests/
|   └─ integration/revocation_enforcement_test.go
|   └─ integration/mitm_detection_test.go
|   └─ e2e/bootstrap_test.sh

```

```
|
|— scripts/{bootstrap.sh,unseal.sh,demo-revoke.sh,teardown.sh}
|— .github/workflows/{ci.yml,security-
scan.yml,release.yml,scorecard.yml}
```

4. Data Flow — 5 luồng, giải thích cách các tool nói chuyện với nhau

Flow A — Bootstrap (**make up**)

1. Compose khởi động Vault (raft) + một "seal vault" nhỏ chứa transit key để auto-unseal. Vault chính unseal bằng transit seal → không cần gõ tay 3 key shard.
2. `scripts/bootstrap.sh` chờ Vault healthy, xuất `VAULT_TOKEN` tạm.
3. `terraform apply` trong `terraform/bootstrap`: tạo mount `pki_root` → generate root (10y) → tạo `pki_int` → CSR → root ký → set-signed. Bật ACME, set `crl_distribution_points` và `ocsp_servers` trong AIA.
4. Terraform apply policy + approle cho từng service. Root token bị revoke ở cuối script — **viết rõ điều này trong README**, đây là điều senior làm mà junior quên.

Flow B — Issuance

- Traefik dùng ACME client trở vào `https://vault:8200/v1/pki_int/acme/directory` → tự lấy cert cho `*.internal`. Public domain thì trở Let's Encrypt staging (mặc định staging để tránh rate limit khi người ta clone về test — chi tiết nhỏ nhưng thể hiện bạn từng bị burn).
- `demo-api` login bằng AppRole → lấy client cert TTL 24h + secret DB từ KV-v2 → renew bằng sidecar/agent template.

Flow C — Revocation Assurance (★ core loop)

1. `revocation-probe` mỗi 15 phút: xin 1 canary cert cho `<uuid>.internal`.
2. Dựng ephemeral TLS listener dùng cert đó.

3. Gọi Vault revoke serial, ghi `t_revoke`.
4. Chạy song song N **client profile** (đây chính là RQ2 matrix của bạn, giờ thành code): `curl` hard-fail, `curl` soft-fail, `openssl s_client -status`, Go `crypto/tls`, Python `requests`, headless Chromium. Mỗi profile poll đến khi reject → ghi `t_detect`.
5. Export:
 - `pki_revocation_detection_seconds{profile,method}` (histogram)
 - `pki_softfail_total{profile}` (counter)
 - `pki_ocsp_responder_latency_seconds`
 - `pki_cert_expiry_seconds{cn}`
6. Prometheus rule: `pki_softfail_total` tăng → Alertmanager → Slack.
Đây là lần đầu SME có SLO cho revocation.

Bonus dùng lại nguyên `tc netem`: thêm mode `--chaos` inject delay vào OCSP path → dashboard vẽ lại đúng cái curve soft-fail threshold ~1960ms bạn đã đo. Cho vào `docs/benchmarks/` kèm chart cũ.

Flow D — Trust drift & MITM detection (RQ2 → product)

- Agent trên endpoint đọc system trust store (macOS keychain / `/etc/ssl/certs` / Windows store), hash SPKI từng root, so với baseline đã ký.
- Root CA lạ xuất hiện → event → Wazuh rule custom → alert "Unauthorized root CA installed on host X". Đây chính xác là detection cho Scenario B trong paper của bạn.
- `ct-watcher` poll CT log cho domain tổ chức → cert lạ được issue → alert.

Flow E — CI/CD gate

```

push → gitleaks (secret leak)
      → checkov + tfLint (IaC misconfig)
      → hadolint (Dockerfile)
      → conftest (OPA: cấm TTL > 90d, cấm wildcard SAN, bắt buộc AIA
có OCSP)
      → docker build
      → trivy image --severity CRITICAL,HIGH --exit-code 1
      → syft → SBOM (SPDX) attach vào release
      → cosign sign --keyless (OIDC từ GitHub) ← keyless signing =
rất ít người biết
      → INTEGRATION TEST: dựng full stack trong CI, revoke cert
thật,
      assert detection_time < 15s VÀ softfail_count == 0
      → push GHCR

```

Cái integration test cuối cùng là thứ khiến pipeline này không phải "trang trí": **CI của bạn test security property, không chỉ test code.** Nói câu đó trong interview.

5. Step-by-step từ con số 0

Chia 6 phase. Đừng làm song song, mỗi phase phải chạy được rồi mới sang phase sau.

Phase 0 — Scaffold (1 ngày)

Repo, LICENSE, Makefile rộng, `docker-compose.yml` chỉ có Vault dev mode. Mục tiêu: `make up` → vào được UI Vault. Commit.

Phase 1 — PKI hierarchy bằng Terraform (3-4 ngày)

Bỏ dev mode, chuyển raft + transit auto-unseal. Viết

`terraform/bootstrap`: root, intermediate, role, AIA/CRL URL. Verify bằng `openssl x509 -text` thấy đúng OCSP URI. Viết ADR-0001 giải thích tại sao chọn Vault thay vì step-ca. **Milestone:** `make up` → có CA đầy đủ, root token bị revoke.

Phase 2 — ACME + Traefik + demo-api (3-4 ngày)

Bật ACME trên `pki_int`. Traefik tự lấy cert. `demo-api` login AppRole, lấy mTLS cert. Milestone: `curl --cacert ca.crt https://api.internal` chạy được, cert TTL 24h tự renew.

Phase 3 — revocation-probe (1-1.5 tuần) ← đầu tư nhiều nhất vào đây

Viết Go service. Bắt đầu với 2 profile (curl hard-fail, Go tls), thêm dần.

Export Prometheus. Viết `scripts/demo-revoke.sh` để người clone gõ 1 lệnh là thấy alert bắn ra — **cái này quay GIF cho README**.

Milestone: Grafana panel hiển thị detection time theo profile, có ít nhất 1 profile soft-fail để chứng minh vấn đề là thật.

Phase 4 — Observability + Wazuh (4-5 ngày)

Prometheus/Grafana/Alertmanager → Slack webhook. Vault audit device → file → Wazuh agent + custom decoder cho JSON audit log. Dashboard 3 cái. Milestone: revoke cert → 2 phút sau có tin nhắn Slack.

Phase 5 — CI/CD + supply chain (4-5 ngày)

Full pipeline như Flow E. Phần khó nhất là integration test dựng stack trong runner — dùng `docker compose up -d` + healthcheck loop, dùng `sleep 60`. Milestone: badge xanh + SBOM trong release + image có cosign signature.

Phase 6 — Docs & polish (1 tuần, đừng cắt)

README chuẩn, threat model STRIDE, 3 runbook, ADR, `docs/benchmarks/` chứa research CYB 260. Demo GIF (dùng `vhs` của Charm để record terminal đẹp).

Tổng ~5-6 tuần làm nghiêm túc. Nếu gấp: Phase 0→3 + README tốt vẫn đủ ấn tượng hơn 90% portfolio.

6. README skeleton

markdown

```
# pki-sentinel
> Internal PKI for SMEs that proves your clients actually reject revoke

[badges: CI | trivy | license | SBOM | OpenSSF Scorecard]

![demo](docs/images/demo.gif)

## Why I built this
[3 đoạn: SME dùng self-signed → cert expire gây downtime → deploy revoke nhưng chưa ai từng verify. Dẫn số liệu thí nghiệm của chính bạn: soft-fail rate đạt 100% khi OCSP latency chậm ~1.96s.]

## Quick Start (3 commands)
git clone ... && cd pki-sentinel
cp .env.example .env
make up          # ~90s, sau đó mở http://localhost:3000

## Architecture      [diagram]
## What it does      [3 plane]
## Try the demo      make demo-revoke → xem alert bắn ra
## Threat model      [link]
## Production notes  [auto-unseal phải dùng KMS thật, root CA nên offline]
## Benchmarks        [link tới research của bạn]
## Roadmap / License / Acknowledgements
```

Section "**Production notes**" — liệt kê những thứ demo này *cố tình* làm không an toàn và cách sửa cho prod. Đây là thứ tách kỹ sư 4 năm khỏi sinh viên: biết giới hạn của chính sản phẩm mình.

7. Cảnh báo thẳng

- **Đừng nhét Kubernetes vào ngay.** Docker Compose chạy được > K8s manifest chạy nửa vời. Để k8s/Helm ở Roadmap.
- **SonarQube bỏ được.** Nó nặng, cần Postgres, và giá trị thấp cho repo Go nhỏ. Thay bằng `golangci-lint` + `gosec` + CodeQL. Nếu recruiter

hỏi "sao không SAST?" → trả lời "CodeQL + gosec, SonarQube overkill cho scope này" là câu trả lời đúng chứ không phải thiếu sót.

- **Rủi ro** 70%. R You are a Principal Software Architect. You fully understand the blueprint of our project: ****pki-sentinel**** (an internal PKI platform for SMEs combined with a continuous revocation assurance plane, inspired by real-world CYB 260 research on OCSP soft-fail).
- **Đừng** tại của trước r duy là

Bắt đầu từ l
PKI hierarc

Show more

Architected comprehensive multi-phase infrastructure blueprint for autonomous exe...

- 🕒 The user needs a comprehensive master plan document as a markdown file. I don't have a specific markdown skill listed, so I'll create it directly without checking additional resources.
Since this is a standalone plan document for a coding agent, a markdown artifact is the right approach. The user requested crystal-clear English for the plan itself, so I'll write it in English while keeping my chat response brief. I need to be technically precise about the Vault configuration details — specifically around transit auto-unseal with a secondary Vault instance, PKI ACME setup requirements for Vault 1.14+, and the necessary cluster path and ACME configuration steps.
Now I'm detailing the OCSP responder configuration, the specific Terraform provider resources needed for the PKI infrastructure, how to integrate Traefik's ACME resolver with the custom Vault CA, and the Raft storage backend setup.
For security scanning and monitoring, I'm considering tools like Trivy, gitleaks, checkov, hadolint, syft, and cosign for keyless signing, plus Wazuh for log analysis with custom decoders to parse Vault audit events. The Wazuh setup would be resource-intensive as a single-node deployment, so I'll provide it as an optional component.
For revocation checking, the OCSP responder should respond quickly when clients perform stapling checks, though I'm reconsidering the best approach for probing revocation status...
Most clients actually don't validate revocation by default—Go's crypto/tls skips it entirely, Python requests doesn't check OCSP, and even headless Chromium with CRLSets won't catch a private CA revocation. That's the honest finding worth demonstrating. I also need to account for OCSP cache TTL through Vault's

`ocsp_expiry` configuration, and chaos testing with network emulation will require NET_ADMIN capabilities.

Now I'm setting up the full documentation structure with a preamble covering global

Show less

📁 Check available skills and output dir

🕒 I'm going to start writing out the master plan for this project.

📄 Master implementation plan for pki-sentinel, written for an autonomous coding agent

📄 Presented file

👌 Done

Master plan xong — Phase 0 → 6, mỗi step có **Target Files / Action / Verification** để feed thẳng cho Claude Code.

Vài điểm t cố ý gài vào, m nên biết lý do trước khi agent chạy:

\$0 Global Contract — đọc kỹ phần này. Nó cấm `sleep`, cấm `:latest`, bắt commit từng step. Nếu không có nó, agent sẽ tạo ra một đống code chạy được trên máy nó và flaky trên CI.

Pre-flight guard (Step 3.5.3) — trước khi revoke, mọi profile phải connect thành công trước. Thiếu bước này thì mọi lỗi network sẽ bị đếm nhầm thành "detection", và toàn bộ số liệu vô nghĩa. Đây là bug hay gặp nhất trong loại harness này.

Bảng client profile (Step 3.4) — kết quả thật sẽ là: `curl` mặc định, Go stdlib, Python requests đều **accept** cert đã revoke. Đó không phải lỗi, đó là lý do project tồn tại. Đưa bảng đó lên README section 9.

Chaos mode (Step 3.7) — chính là RQ1 của m, nhưng thành feature có Prometheus metric thay vì lab một lần. Dải delay dày quanh 1950–2000ms vì bài của m tìm ra vùng chuyển tiếp nhiều chứ không phải ngưỡng sạch.

Alert chỉ bắn khi `method != "none"` (Step 4.2) — client vốn không check revocation thì đừng page. Chi tiết tuning này là thứ phân biệt hệ thống thật với demo, và ADR-0007 giải thích nó.

Production notes table (Step 6.4) — liệt kê thẳng những chỗ demo cố tình làm không an toàn. Đừng bỏ. Recruiter nào cũng sẽ tự tìm ra chúng; có sẵn câu trả lời thì thành điểm cộng.

Hai chỗ agent dễ kẹt nhất, t đã ghi vào Appendix C: transit auto-unseal token (Step 1.1) và ACME 404 do thiếu mount tune headers (Step 2.1). Nếu nó loop ở đó thì dừng, làm tay một lần, rồi ghi lại thành script.

Chạy Phase 0 + 1 trước rồi paste `terraform/bootstrap/` lên đây, t review hierarchy cho.



Pki sentinel master plan

Document · MD

[Download](#)

